Davante

Práctica 1

Entornos de Desarrollo
1º Desarrollo de Aplicaciones Multiplataforma

Profesorado: Lorenzo Toro

Curso 2026/2027

Trabajo presentado por:
Francisco Javier Garrido Fernández

Davante

Índice

Práctica 1: Pseudocódigo y paradigmas....3
1. El caso....3
2. Actividades....3
Actividad 1: La calculadora de medias que no se rompe....3
Actividad 2: Un algoritmo con bucle: el contador de tareas....4
Actividad 3: Los paradigmas de programación....5
3. Entrega....5

Práctica 1: Pseudocódigo y paradigmas

1. El caso

La clase va a estrenar la **app de gestión del aula**, un pequeño programa para que el delegado lleve la cuenta de las tareas del grupo y las notas de los retos.

Antes de escribir ni una línea de código hay que **pensar el programa**: qué datos entran, qué pasos se siguen y qué sale por pantalla.

2. Actividades

Actividad 1: La calculadora de medias que no se rompe

El programa de la app debe calcular **la media de las notas** de un reto. Tu compañero/a escribe este pseudocódigo, pero tiene un fallo: si alguien no tiene notas, la suma es 0 y al dividir entre 0 el programa **se rompe** (o muestra un resultado absurdo):

```
INICIO
    LEER numeroNotas
    suma = 0
    contador = 0
    SI numeroNotas == 0 ENTONCES
    IMPRIMIR “No se puede calcular.”
    SI NO
    MIENTRAS contador < numeroNotas HACER
        LEER nota
        suma = suma + nota
        contador = contador + 1
    FIN MIENTRAS
    media = suma / numeroNotas
    ESCRIBIR media
    FIN SI
FIN
```

Se pide:

- Reescribe el pseudocódigo **usando una condicional** para que, si el número de notas es 0 (el divisor), **muestre un mensaje de error al usuario** en lugar de hacer la división.
- Comprueba tu pseudocódigo con la **prueba de escritorio** de los cuatro casos de la tabla. La prueba de escritorio consiste en **ejecutar el algoritmo a mano**, como si tú fueras el ordenador: recorre las instrucciones en orden, anota el valor de las variables y lo que se mostraría en pantalla, tal y como hemos hecho en clase. Es la forma de comprobar si un algoritmo funciona antes de programarlo.

Caso de prueba	¿Qué debe mostrar el programa?	Tu prueba: ¿qué mostró?
3 notas (7, 8, 9)	8	8
2 notas (1, 4)	2	2
1 nota (4)	4	4
0 notas	No se puede calcular.	No se puede calcular.

Actividad 2: Un algoritmo con bucle: el contador de tareas

Otra parte de la app **cuenta las tareas del grupo**. El delegado introduce por teclado el estado de cada tarea: **T** si está terminada o **P** si está pendiente. Al final hay que decir cuántas hay de cada tipo.

Escribe el **pseudocódigo completo** de este algoritmo. Debe:

- Leer los estados de las tareas **hasta que** se escriba TERMINAR (bucle).
- Contar cuántas son **T** y cuántas son **P**.
- Al terminar, mostrar el **total de tareas**, las terminadas y las pendientes.

```
INICIO
        t = 0
        p = 0
        respuesta = " "

        MIENTRAS respuesta NO ES "TERMINAR" HACER
                IMPRIMIR "Introduce T (tarea terminada) o P (tarea pendiente). Para salir, introduce TERMINAR:"
                LEER respuesta

                SI respuesta == "T" ENTONCES
                t = t + 1
                FIN SI

                SI respuesta == "P" ENTONCES
                p = p + 1
                FIN SI

        FIN MIENTRAS

totalTareas = t + p
IMPRIMIR "Tareas terminadas:" + t
```



```
IMPRIMIR “Tareas pendientes:” + p
IMPRIMIR “Total de tareas:” + totalTareas

FIN
```

Actividad 3: Los paradigmas de programación

En el tema hemos visto que los **dos grandes enfoques** de los paradigmas son el **imperativo** (decir cómo se hace, paso a paso) y el **declarativo** (decir qué quieres conseguir). Pero no son los únicos: existen **más paradigmas**, y muchos lenguajes actuales son **multiparadigma** porque combinan varios.

Debes investigar **2 de los siguientes 6 paradigmas**: orientado a objetos, funcional, estructurado, lógico, orientado a eventos y concurrente.

Para **cada paradigma elegido**, busca en internet y anota:

- Una **definición con tus palabras** (máximo 4 líneas): qué idea de programación defiende.
- Un **ejemplo de lenguaje** que lo utilice e indica si además lo combina con otros.
- Un **pequeño trozo de código** en ese lenguaje (5-10 líneas) que muestre su estilo.
- Las **direcciones web** donde has encontrado la información: todo lo que afirmes debe poder comprobarse.

1) Orientado a objetos: el programa cuenta con unas clases que describen el problema o concepto a tratar y lo divide en distintos objetos que colaboran entre sí para solucionarlo mediante métodos. Un lenguaje multiparadigma que sigue este ejemplo sería Java, que puede ser imperativo al mismo tiempo también.

```java
public class Hand {
    private int value;
    private int upperBound;

    public Hand(int upperBound) {
        this.upperBound = upperBound;
        this.value = 0;
    }
```

Fuentes:
https://java-programming.mooc.fi/part-7/1-programming-paradigms
https://cs.lmu.edu/~ray/notes/paradigms/

2) Funcional: el problema planteado se resuelve mediante funciones. No hay estados que modifiquen el comportamiento interno de las mismas, como sí ocurre con los métodos de los objetos en la POO. En su lugar, cada función depende de sus propias entradas y cómo estén planteadas. Haskell es un lenguaje que sigue este paradigma, siendo funcional puro.

```
readsTree              :: (Read a) => ReadS (Tree a)
readsTree s            =  [(Branch l r, x) | ("<", t) <- lex s,
                                             (l,   u) <- readsTree t,
                                             ("|", v) <- lex u,
                                             (r,   w) <- readsTree v,
                                             (">", x) <- lex w         ]
                          ++
                          [(Leaf x, t)     | (x,   t) <- reads s       ]
```

Fuentes:
https://docs.python.org/3/howto/functional.html
https://www.haskell.org/
https://www.haskell.org/tutorial/stdclasses.html (captura de pantalla del código de ejemplo).

Para terminar, responde: ¿el pseudocódigo de las actividades 1 y 2 es **imperativo** o **declarativo**? Justifica tu respuesta en 3 líneas como máximo.

El pseudocódigo de las actividades 1 y 2 es imperativo porque le indicamos a la máquina cómo debe ejecutar el programa, y lo hace paso a paso, línea a línea de código.

3. Entrega

La práctica se debe elaborar **de forma individual**. Deben realizarse **todas** las actividades planteadas en este documento. Cada actividad debe comenzar en una página nueva.

Se entregará a través del aula virtual en **formato PDF**, utilizando **este mismo documento** como plantilla. No se aceptarán entregas fuera de plazo ni por email.

En la **portada** debe figurar **tu nombre**. Recuerda citar las fuentes web en la actividad 3.

Debes escribir las respuestas a las actividades en **color azul**.